

7.1

```
// Use Euclid's algorithm to calculate the GCD.  
// See en.wikipedia.org/wiki/Euclidean_algorithm.  
private long GCD(long a, long b)  
{  
    a = Math.Abs(a);  
    b = Math.Abs(b);  
    for (; ; )  
    {  
        long remainder = a % b;  
        if (remainder == 0) return b;  
        a = b;  
        b = remainder;  
    };  
}
```

Here I removed the unnecessary comments to make the code looks cleaner and avoid restating what the code have said. In addition, I also added 2 comments to state the content of the code (Euclid's algorithm) and the reference.

7.2

The 2 conditions that might ended up with bad comments in the previous code are:

1. Top Down Design

- a. Describing the code in excruciating detail that the explanation become redundant because the comment describes a code statement.
- b. Solution to this is to stop one step before the actual code.

2. Adding comments after writing code

- a. After the code is written it's easy to just say what each line of code does and why it is doing it.
- b. Solution is to write comments before coding.

7.4

```

// Use Euclid's algorithm to calculate the GCD.
// See en.wikipedia.org/wiki/Euclidean_algorithm.
private long GCD(long a, long b)
{
    if (a <= 0)
        throw new ArgumentException("Parameter 'a' must be greater than 0", nameof(a));
    if (b <= 0)
        throw new ArgumentException("Parameter 'b' must be greater than 0", nameof(b));

    long original_a = a;
    long original_b = b;

    for (; ; )
    {
        long remainder = a % b;
        if (remainder == 0)
        {
            if (original_a % b != 0)
                throw new InvalidOperationException(
                    $"GCD result {b} does not divide original value a={original_a}");
            if (original_b % b != 0)
                throw new InvalidOperationException(
                    $"GCD result {b} does not divide original value b={original_b}");

            return b;
        }
        a = b;
        b = remainder;
    };
}

```

In this modified code it is more offensive because it use throw exceptions for invalid input and it also verify the result with exceptions.

7.5

Since I'm using throw exceptions in previous code I don't need to add error handling code anymore because exceptions is already passing it up to the calling code so they can be handled there.

7.7

Top-down design is to start with a high-level statement of a problem and break the problem down into more detailed pieces. Here we make it like this:

DriveToSupermarket()

1. PrepareToLeave()

- a. Locate car keys.
- b. Get inside the car.

- c. Sit in the driver seat.
- 2. StartCar()
 - a. Press engine start button.
- 3. NavigateToSupermarket()
 - a. Open Google Map to check the closest route to supermarket
 - b. FollowGPS()
- 4. ParkCar()
 - a. Find parking
 - b. Pull the car into the space
 - c. Shift gear to Park
 - d. Press engine stop button

FollowGPS()

- 1. TrafficLights()
- 2. Turn()
- 3. DriveStraight()

TrafficLights()

- 1. If (traffic light is red) → stop behind the line and wait.
- 2. If (traffic light is yellow) → slow down and prepare to stop.
- 3. If (traffic light is green) → proceed through intersection.
- 4. If (stop sign present) → come to a full stop, check for cross traffic, then proceed.
- 5. If (yield sign present) → slow down and give way to oncoming traffic before proceeding.

Turn()

- 1. Check mirrors for surrounding traffic.
- 2. Activate the appropriate turn signal (left or right).

3. Slow to a safe turning speed.
4. Check for pedestrians and oncoming vehicles.
5. If (all clear) → steer through the turn into the correct lane.
6. Cancel turn signal after completing the turn.

Assumptions

- The driver has a valid license.
- The car has fuel.
- The driver starts from home.

8.1

Here is the test for isRelativelyPrime()

```

using System;
using System.Diagnostics;

public void Test_IsRelativelyPrime()
{
    Random rand = new Random();

    for (int i = 0; i < 1000; i++)
    {
        int a = rand.Next(-1000000, 1000001);
        int b = rand.Next(-1000000, 1000001);

        if (a == 0 || b == 0) continue;

        Debug.Assert(
            IsRelativelyPrime(a, b) == IsRelativelyPrime2(a, b),
            $"Failed for ({a}, {b})"
        );
    }

    Console.WriteLine("All tests passed!");
}

private bool IsRelativelyPrime2(int a, int b)
{
    a = Math.Abs(a);
    b = Math.Abs(b);

    if ((a == 1) || (b == 1)) return true;
    if ((a == 0) || (b == 0)) return false;

    int min = Math.Min(a, b);
    for (int factor = 2; factor <= min; factor++)
    {
        if ((a % factor == 0) && (b % factor == 0))
            return false;
    }
    return true;
}

```

So Test_isRelativelyPrime() is to test 1000 random number pairs to test if the result from isRelativelyPrime() match with isRelativelyPrime2(). isRelativelyPrime2() is the method that I

made which is the same as `isRelativelyPrime()`. Therefore, if the result match then the method is correct.

8.3

In the previous problem I use Black-Box testing technique because it just said the suppose you have written an efficient `isRelativelyPrime` method but it doesn't say how, so I use Black-Box technique which is testing based on input and expected output.

White-Box technique is used when we know the internal implementation, Grey-Box is used when we have partial knowledge of the implementation and Exhaustive is used if the input range is small enough to test the combination.

8.5

The bug is in `Math.Abs(int.MinValue)` which causes integer overflow. Since the minimum int is $= -2,147,483,648$ and Positive int $= +2,147,483,647$ there is no positive equivalent for the minimum. The solution is to restrict the value to $-1,000,000$ to $+1,000,000$.

Testing the code does help as it show the bugs. It shows the integer overflow and it also made me to add some input restrictions to make the code works better.

8.9

Exhaustive testing falls into black-box test because they don't rely on knowledge of what's going on inside the method they are testing.

8.11

This problem is using this formula to calculate bugs. $Bugs = (E1 \times E2) : S$

$E1 = \# \text{ bugs found by tester 1}$

$E2 = \# \text{ bugs found by tester 2}$

$S = \# \text{ bugs found by both tester}$

$\text{Alice-Bob(Lincoln Index)} = (5 \times 4) : 2 = 10$

$\text{Alice-Carmen(Lincoln Index)} = (5 \times 5) : 2 = 12.5$

$\text{Bob-Carmen(Lincoln Index)} = (4 \times 5) : 1 = 20$

$\text{Avg bugs} = 10 + 12.5 + 20 = 42.5 : 3 = 14$

$\text{Worst Case} = 20$

$\text{Union} = \{1, 2, 3, 4, 5, 6, 7, 8, 9, 10\} = 10$

Bugs still at large:

$$\text{Avg} = 14 - 10 = 4$$

$$\text{Worst case} = 20 - 10 = 10$$

Plan for worst case 20 bugs but expect the avg which is 14 bugs.

8.12

If the testers don't have any bugs in common, then the equation for the lincoln index divide by 0, which result in infinity (don't know how many bugs are there). We can get the lower bound for the number of bugs by pretending they have 1 bug in common so the equation will be divided by 1.